# Machine Learning

## Nearest Neighbor Classification

Hoda Eldardiry

# Lecture Objectives

- Explain how Nearest Neighbor and KNN work
- Describe how distance metrics affect predictions
- Reason about the bias–variance trade-off using K
- Correctly use train / validation / test splits
- Explain overfitting vs underfitting *with KNN as an example*

# Key Idea

- **Design Question:**
    - How can we classify a new data point if we don’t build an explicit model?
- **Key idea:**
    - Sometimes similar examples have similar labels
    - Instead of learning parameters, we **store the data** and compare
- **Real-world intuition:**
    - Movie recommendations
    - Handwritten digit recognition
    - Medical case similarity

# Nearest Neighbor (NN) Classifier

- **Definition**
  - Given a training set of labeled examples:
    - Input: new point *x*
    - Find *x**: closest training point to *x*
    - Output: label of *x**
  - *h(x) = y(x*)*
- **Key Components**
  - Distance metric
  - Stored training data

# Distance Measures

- Find $x^*$: closest training point to $x$ $\quad x^* = argmin_{x'}\, d(x, x')$
- *Closest: smallest distance according to some distance measure d*
- **Distance Measures** $d(x, x')$
  - *L1 (Manhattan): grid-like distance* $\quad L_1: d(x, x') = \sum_j^M |x_j - x'_j|$
  - *L2 (Euclidean): straight line distance* $\quad L_2: d(x, x') = \left(\sum_j^M |x_j - x'_j|^2\right)^{1/2}$
  - ...
  - *Lp: generalization* $\quad L_p: d(x, x') = \left(\sum_j^M |x_j - x'_j|^p\right)^{1/p}$
- **Weighted Distance** $\quad d(x, x') = \left(\sum_j^M c_j |x_j - x'_j|^p\right)^{1/p}$
  - Some features matter more than others

weight

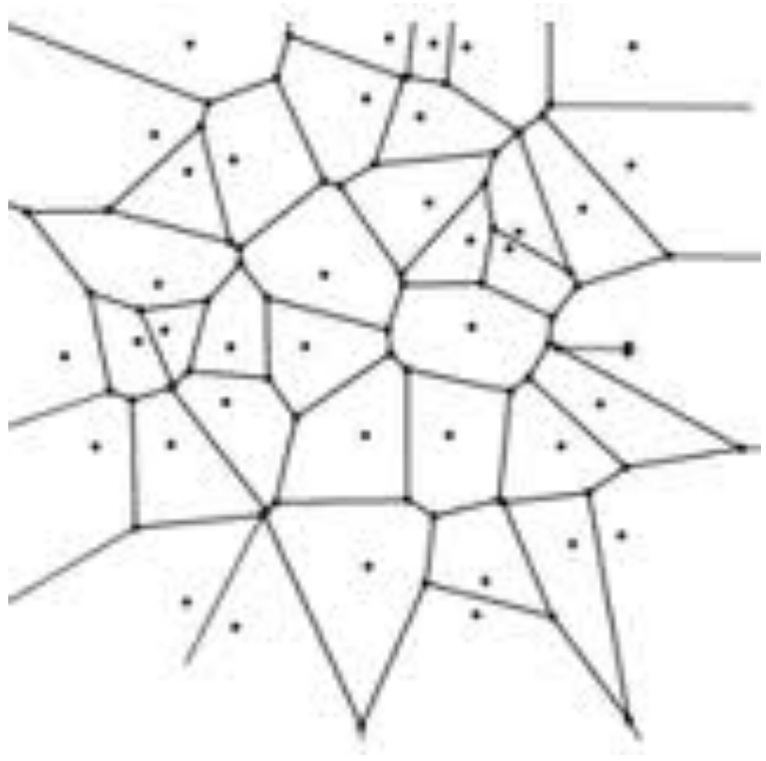
# Distance Measures

- **Key idea:**
  - Sometimes similar examples have similar labels
  - ***The choice of distance defines similarity***
- **Examples:**
  - *Images → Euclidean often works*
  - *Text → Cosine similarity may be better*

# Geometric Interpretation

- **Key idea:** NN induces a partition of the input space
  - Each training point “owns” a region
  - All points in that region get its label
- **Voronoi diagram** (Euclidean distance):
  - Boundaries are straight lines
  - At midpoints between training examples
- **Conceptual takeaway:**
  - NN classifiers are **highly flexible**

Example Partitioning: Voronoi diagram



# Strengths & Weaknesses of NN

- **Pros**
  - Extremely simple
  - No training time
  - Works surprisingly well
- **Cons**
  - Sensitive to noise (unstable)
  - Sensitive to outliers
  - Expensive at prediction time
- **Question:** what happens if one point is mislabeled?

# From NN to KNN

- **Motivation:** Reduce sensitivity to noise
- **KNN idea:**
  - Look at the **k nearest neighbors**
  - Predict the **majority label**
    - $h(x)=\text{mode}(y \in kNN(x))$
- **Special case:**
  - NN = 1-NN

# Why KNN Is More Robust

- **Noise scenario:**
  - One bad point → NN fails
  - Majority vote → KNN recovers
- **Key intuition:**
  - Stability increases as K increases
- But… nothing is free 👀

# The Choice of K

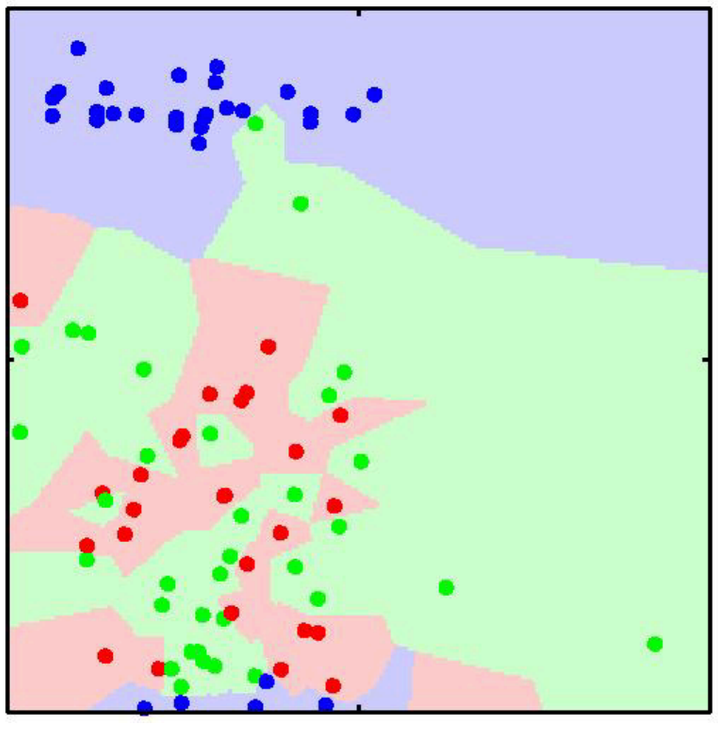
- **Small K (e.g., K=1)**
  - Very flexible
  - Low bias, high variance
  - Overfits
- **Large K (e.g., K=31)**
  - Very smooth boundaries
  - High bias, low variance
  - Underfits
- **Extreme case**
  - K = dataset size → always predict majority class

# *K and Decision Boundaries*

- Which K generalizes better?

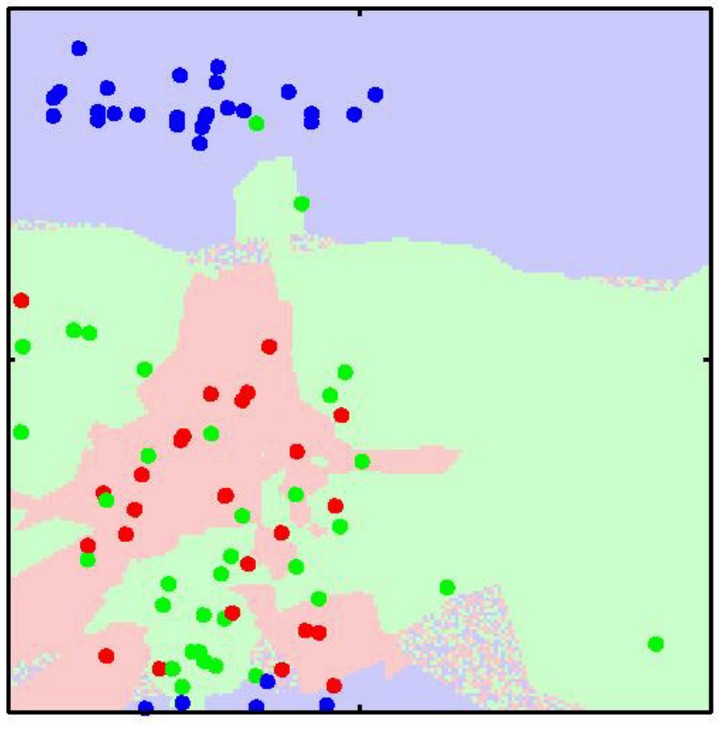
**Voronoi regions**

$K = 1$



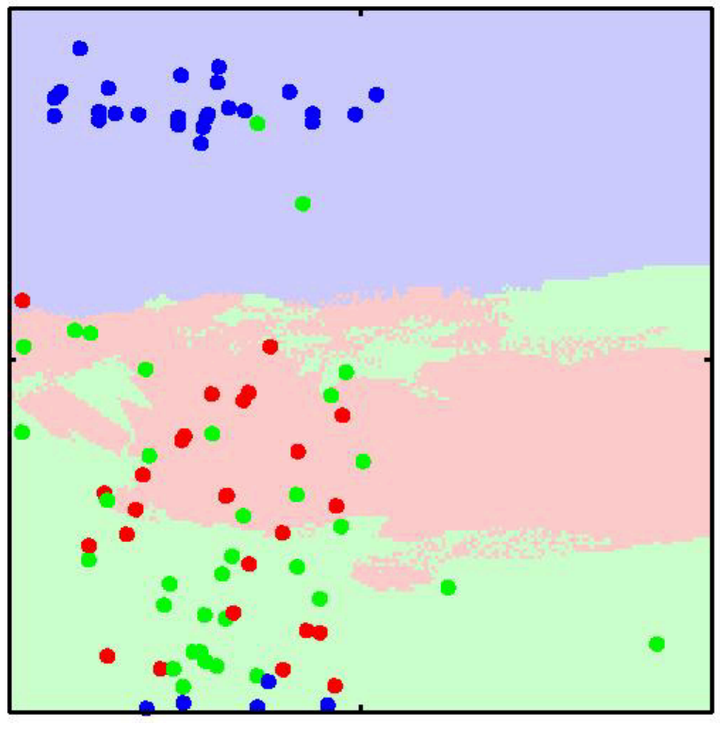
**Smoother boundaries**

$K = 3$



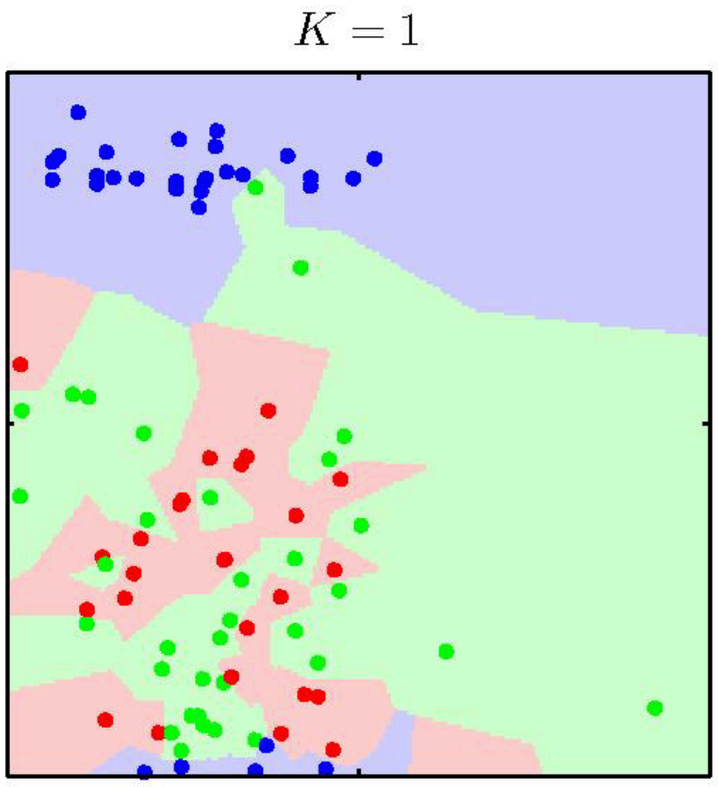
**Overly smooth, unintuitive regions**

$K = 31$



# *K=1*

- Voronoi diagram (just not showing every single region)
- Just showing region color corresponding to 3 classes: blue, green, red
- Merging regions of same class (color)



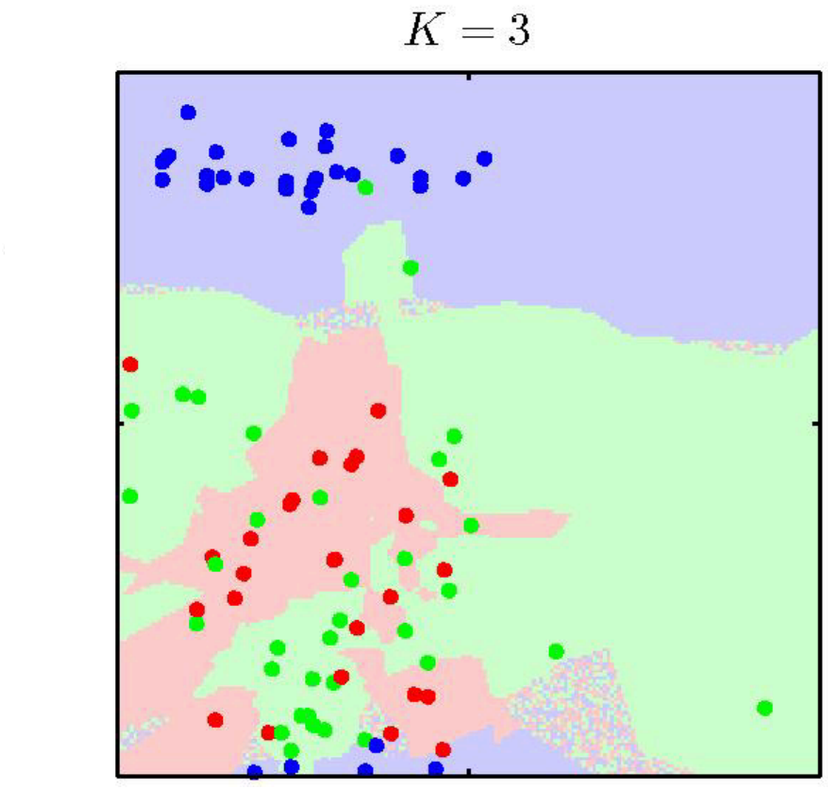

**Intuitively**

— Each red point is surrounded by a red region

— Because this red point will be NN for a point in that region

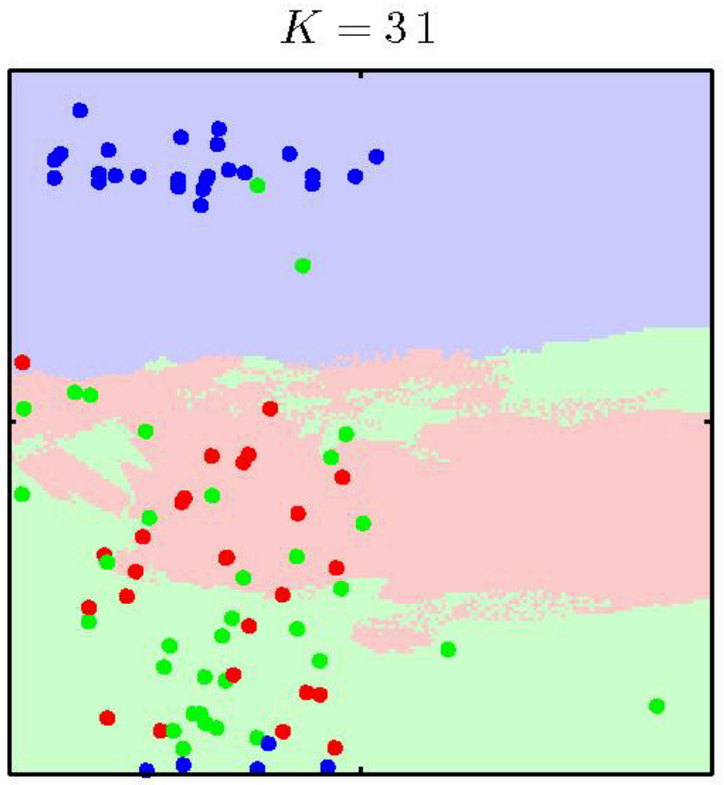
# *K*=3

- **A little less intuitive**
  - A red point in a green region
  - Because considering 3 NN of the red dot can include 1 red & 2 green
  - Majority green leads to a green region



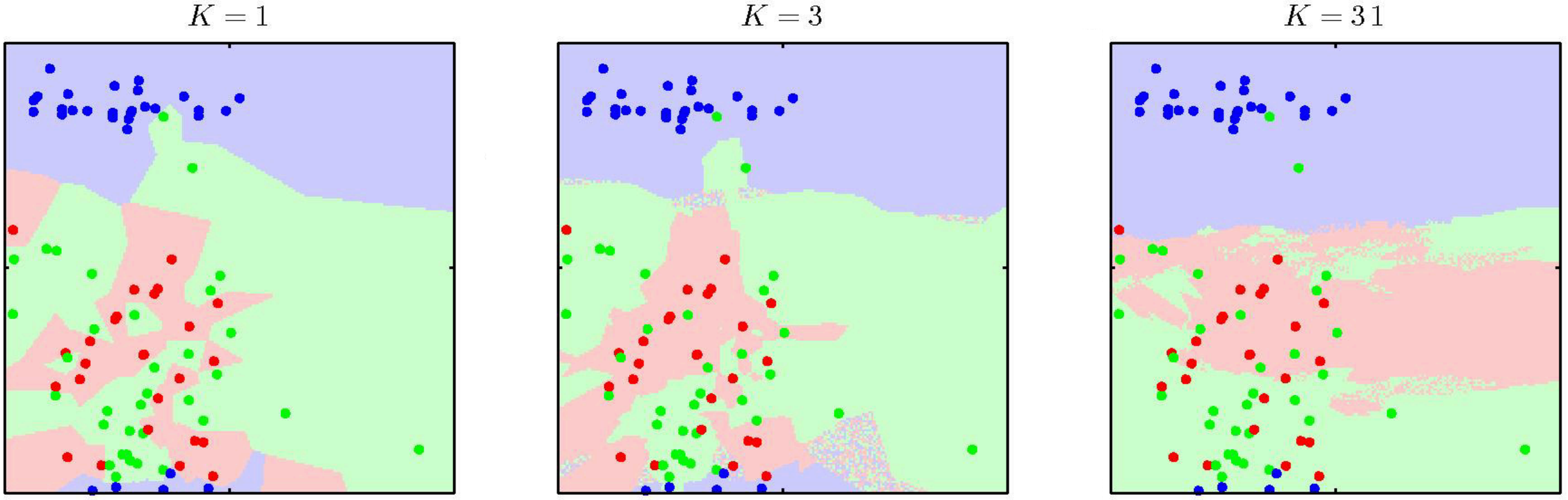
# *K=31*

- **Even less intuitive**
  — Complex boundaries



# *Choice of K!*

- Implicit partitioning arises
- These partitions are just for illustration (not computed using KNN)
- Who thinks K=3 is better?
- Who thinks K=31 is better?



# *Choice of K!*

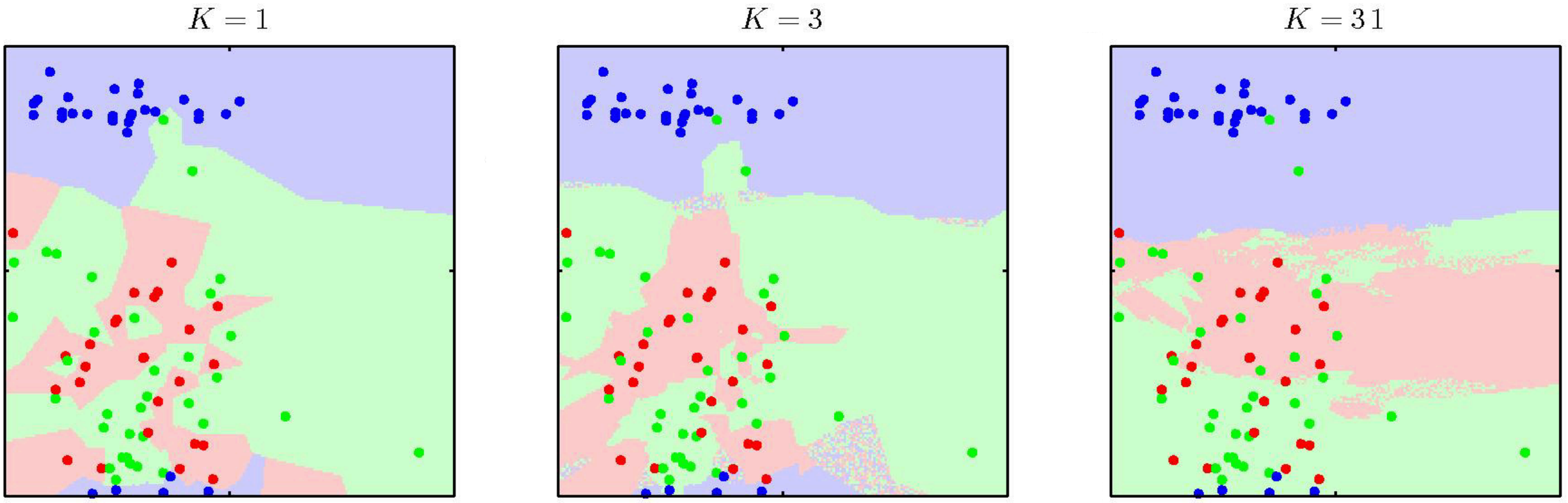
- **No easy answer**
  - Different parts of the pictures suggest different choices
  - It is **problem dependent**

- We need a way to decide
  - K=3 not enough for stability?
  - K=31 too many neighbors?

- Very big K (=dataset size) not good
  - All points will be neighbors
  - All get same classification:
  - Most frequent class in dataset

- Very small K not good
  - Not enough for stability

- Trade-off?



# Measuring Performance

- Goal: predict unseen data well
- Standard workflow:
    - Split data into training and test
    - Train on training set
    - Evaluate on test set
- Important warning:
    - *Training accuracy is misleading*

# Effect of *K* on accuracy

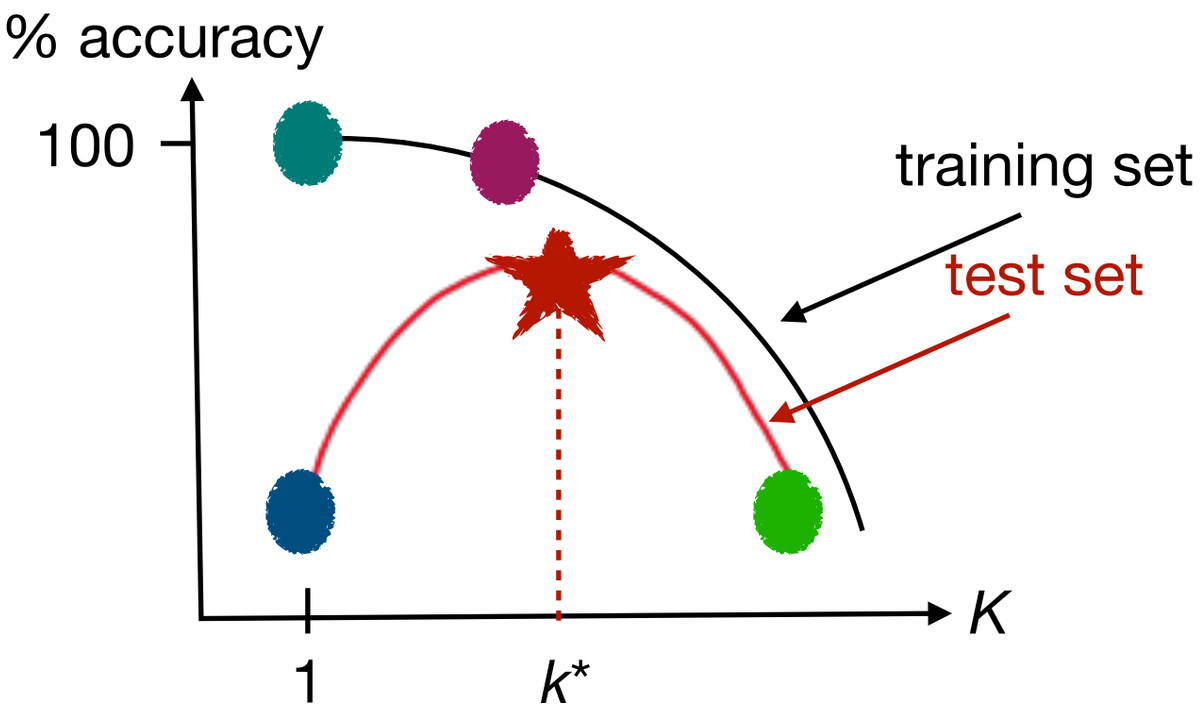
- Best *K* depends on
  - Problem
  - Amount of training data

**Perfect accuracy evaluating on training at k=1**

- Every eval point is in our training set
- NN (point) is the point itself
- Return correct label of that point

**Increasing K reduces accuracy on training**

- Return frequent label of k nearest points




**Poor performance at K=1 on test set**

- Due to noise (can't generalize)

**Optimal performance when K increases**

- Due to increased stability

**Poor performance at large K on test set**

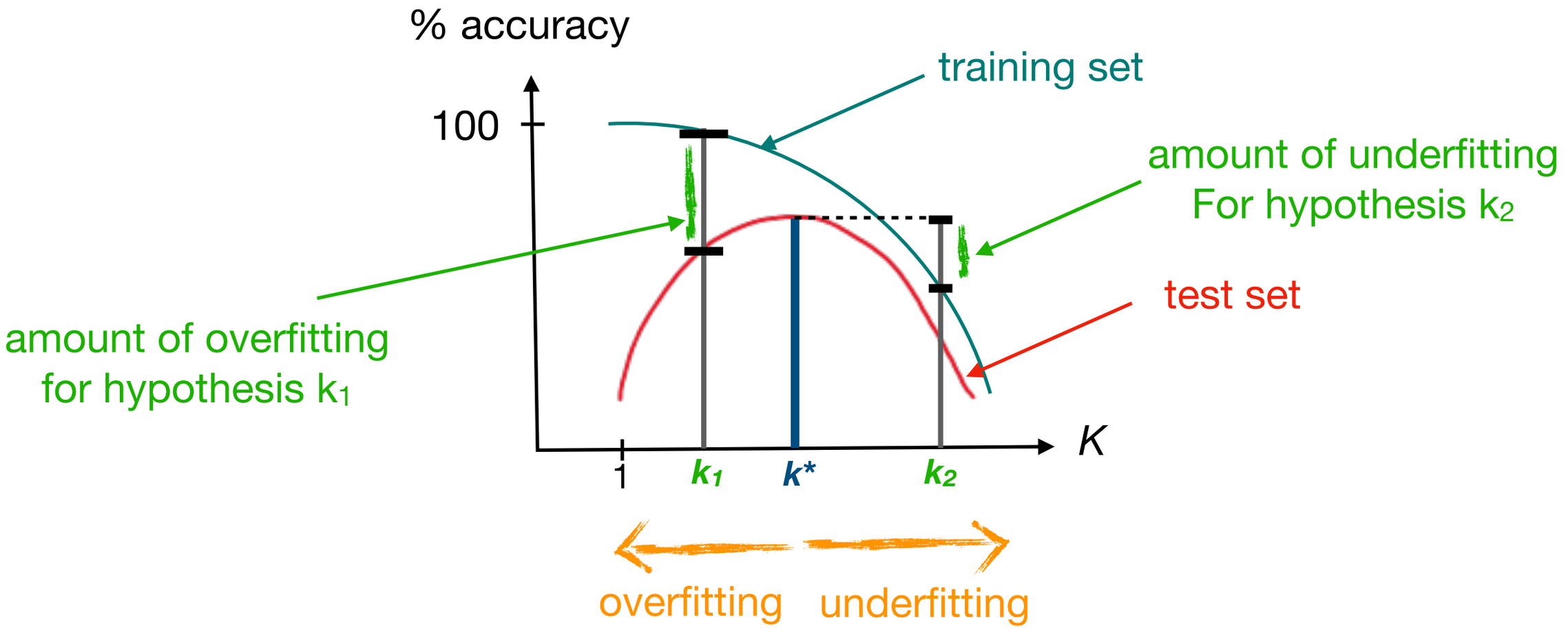
- Use far neighbors (no predictive power)

# Effect of *K* on accuracy

- **Typical pattern:**
  - Training accuracy highest at K=1
  - Test accuracy peaks at some K = K*
- **Interpretation:**
  - Small K → overfitting
  - Large K → underfitting
- *This is the bias–variance trade-off*

# Overfitting vs Underfitting

- **Overfitting**
    - Excellent training accuracy
    - Poor test accuracy
    - Model too complex
- **Underfitting**
    - Poor training accuracy
    - Poor test accuracy
    - Model too simple
- *KNN is a perfect example of this trade-off*


% accuracy
100
training set
amount of underfitting
For hypothesis $k_2$
test set
amount of overfitting
for hypothesis $k_1$
K
1
$k_1$
$k^*$
$k_2$
overfitting
underfitting

# How Do We Choose K?

- **Wrong approach:**
  - Pick K that maximizes test accuracy
  - Why wrong?
    - Test set leaks into training decisions
- **Correct approach:**
  - Training set → fit model
  - Validation set → choose K
  - Test set → final evaluation

# Choosing $K$ based on Validation Set

Let $k$ be the number of neighbours

For $k = 1$ to max # of neighbours

$$h_k \leftarrow train(k, trainingData)$$

$$accuracy_k \leftarrow test(h_k, validationData)$$

$k^* \leftarrow argmax_k \; accuracy_k$

$h \leftarrow train(k^*, trainingData \cup validationData)$

$accuracy \leftarrow test(h, testData)$

Return $k^*, h, accuracy$

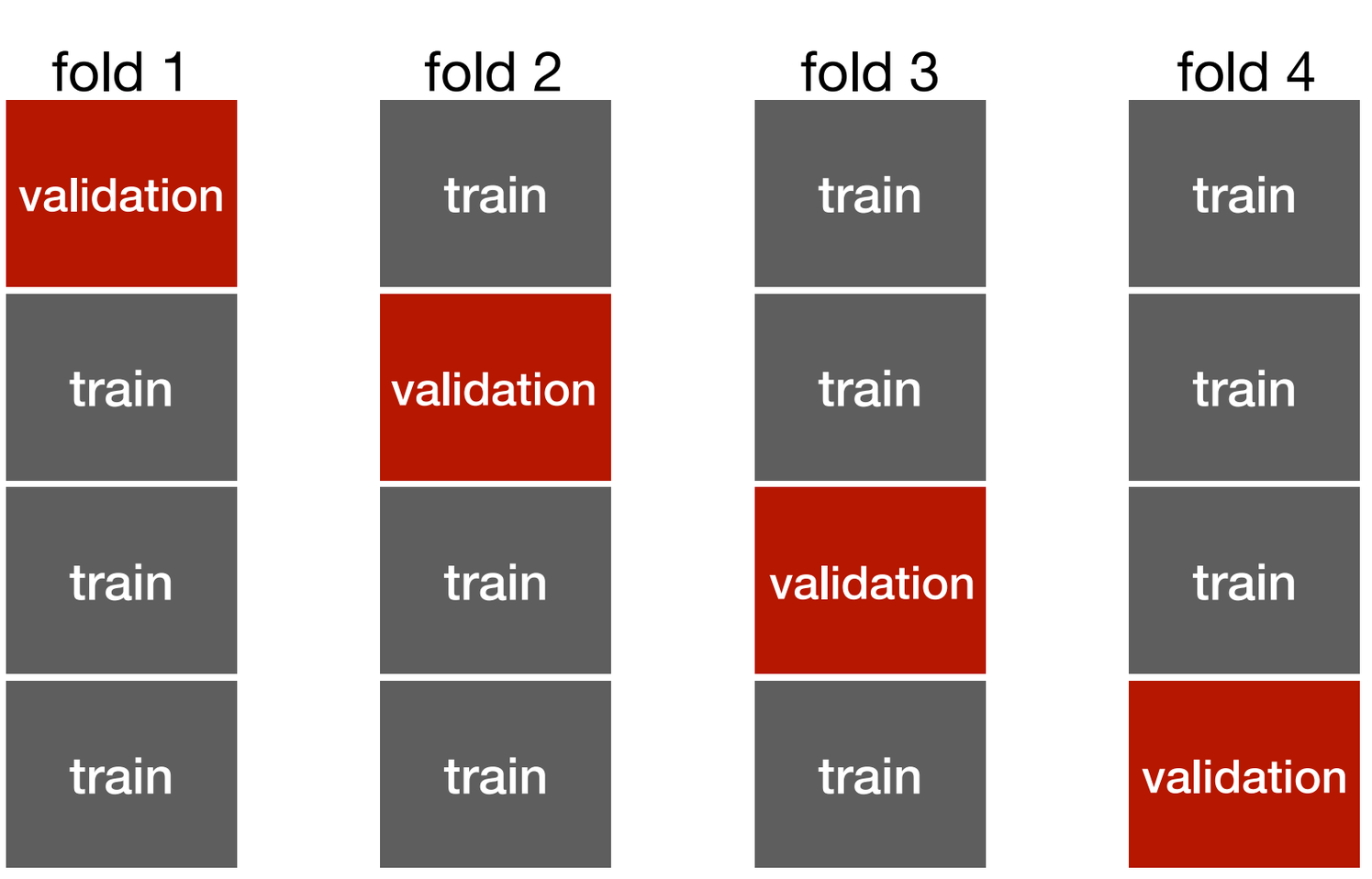
# Cross-Validation

- **k-fold cross-validation:**
  - Split training data into k folds
  - Train on k–1 folds
  - Validate on remaining fold
  - Average validation accuracy
- **Benefits:**
  - More reliable estimate
  - Efficient use of data

# Cross-Validation

- Repeatedly split training data in two parts
  - One for training and one for validation
  - Report the average validation accuracy

**4-fold cross validation**




***k*-fold cross validation**

- Split training data in ***k*** equal size subsets

- Run ***k*** experiments,
  - Each time validate on 1 subset
  - Train on remaining ***k-1*** subsets

- Compute average validation accuracy of the ***k*** experiments

# Selecting #Neighbors by Cross-Validation

Let $k$ be the number of neighbours
Let $k'$ be the number of trainingData splits
For $k = 1$ to max # of neighbours
    For $i = 1$ to $k'$ do  (where $i$ indexes trainingData splits)
        $h_{ki} \leftarrow train(k, trainingData_{1..i-1,i+1..k'})$
        $accuracy_{ki} \leftarrow test(h_{ki}, trainingData_i)$
    $accuracy_k \leftarrow average(\{accuracy_{ki}\}_{\forall i})$
$k^* \leftarrow argmax_k\ accuracy_k$
$h \leftarrow train(k^*, trainingData_{1..k'})$
$accuracy \leftarrow test(h, testData)$
Return $k^*, h, accuracy$

# Weighted K-Nearest Neighbor

- We can often improve K-nearest neighbours by weighting each neighbour based on some distance measure

$$w(x, x') \propto \frac{1}{distance(x, x')}$$

- Label

$$y_x \leftarrow argmax_y \sum_{\{x' | x' \in knn(x) \wedge y = y_{x'}\}} w(x, x')$$

where $knn(x)$ is the set of $K$ nearest neighbours of $x$

*closer neighbors count more*

# K-Nearest Neighbor Regression

- We can also use KNN for regression
- Let $y_x$ be a real value instead of a categorical label

- K-nearest neighbour regression:

$$y_x \leftarrow average(\{y_{x'} | x' \in knn(x)\})$$

- Weighted K-nearest neighbour regression:

$$y_x \leftarrow \frac{\sum_{x' \in knn(x)} w(x, x') y_{x'}}{\sum_{x' \in knn(x)} w(x, x')}$$

*average instead of majority vote*

# Key Takeaways

- KNN is simple but powerful
- Distance metric matters
- K controls bias–variance trade-off
- Validation is essential for hyperparameter tuning
- *KNN trades simplicity for flexibility, and K controls how much we trust local vs global structure*